MULUNGUSHI UNIVERSITY

ICT261 | Unit 4 Java Lab: Exception Handling and Error Management

Name: Makungu Thinkwell Student No.: 202500822

Activity A: Predict and explain (5 marks)

Statement	Exception	Checked / Unchecked	Prevention / handling strategy
Integer.parseInt("ABC");	NumberFormatException	Unchecked (extends IllegalArgumentException, which extends RuntimeException)	Validate the text first (e.g. a regex such as \d+), or wrap the call in try/catch (NumberFormatException) and report a friendly message.
String name = null; name.length();	NullPointerException	Unchecked (extends RuntimeException)	Check for null before use (if (name != null)) or use Objects.requireNonNull() / initialise the variable properly.
new java.io.FileReader("missing.txt");	FileNotFoundException	Checked (extends IOException, which extends Exception)	Check that the file exists first (new File(name).exists()), or wrap in try/catch (FileNotFoundException) / declare throws IOException.

Why OutOfMemoryError should not normally be caught just to continue: OutOfMemoryError is an Error (not an Exception), signalling that the JVM has run out of heap, so the program is in an unreliable state and any attempt to continue will probably fail again or leave data half-updated. The correct fix is to reduce memory use or increase the heap size, not to catch the error and carry on.

Activity B: Safe importer (12 marks)

Full source is in MarksImporter.java (included in the zip). Key points of the implementation:

TODO 1: `if (parts.length != 2) throw new IllegalArgumentException(...)`

TODO 2: `Integer.parseInt(markText)` throws NumberFormatException for text such as "ABC"; a range check (mark < 0 || mark > 100) then throws IllegalArgumentException.

TODO 3: `if (!seen.add(id)) throw new DuplicateStudentException(id);` where DuplicateStudentException extends RuntimeException.

TODO 4: `System.out.println("Accepted: " + id + " -> " + mark);`

Decision: skip or stop on an invalid record?

I chose to skip the invalid record and continue with the rest of the file. Each line is validated inside its own try/catch within the loop, so a non-numeric mark, an out-of-range mark, a wrong number of fields or a duplicate student number is reported to the user in a friendly message, written to the log file with its technical cause, and the importer then moves on to the next line. This suits a university import where one typing error should not prevent hundreds of valid marks from being processed, and the summary line at the end ("2 accepted, 2 skipped") tells the user how many records need correcting. A missing or unreadable file is different: nothing can be imported, so that exception is allowed to propagate out of importMarks() to

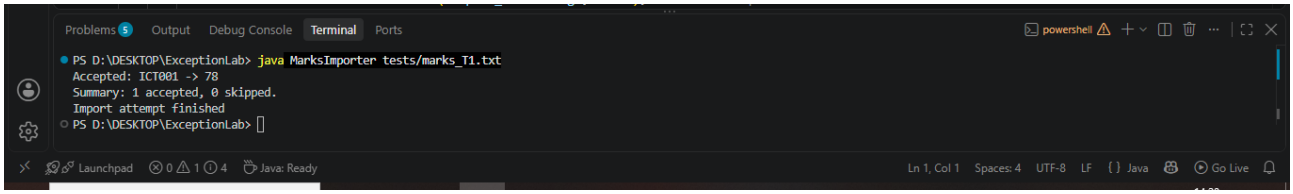
main(), where the import stops with a friendly message. A duplicate ID is also skipped (the first valid record is kept) rather than overwritten, so earlier data is never silently replaced.

Activity C: Handle, log and test (8 marks)

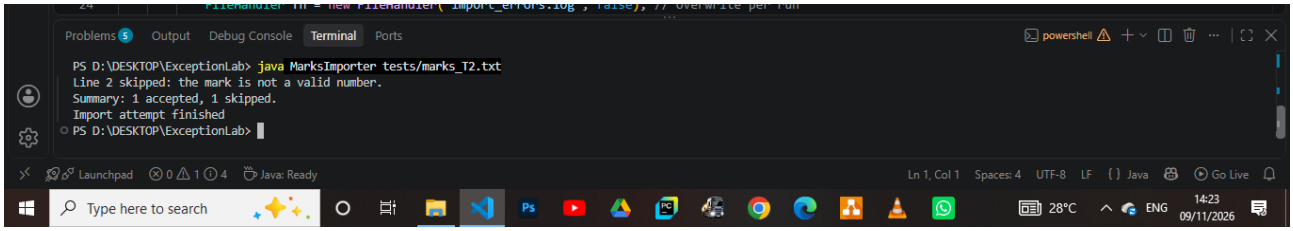
main() calls importMarks inside try/catch with the order FileNotFoundException, IOException, DuplicateStudentException, IllegalArgumentException, then a finally block that prints "Import attempt finished". A java.util.logging.Logger with a FileHandler writes the exception type and technical cause (plus stack trace) to import_errors.log; the console handler is disabled so users never see raw stack traces. Try-with-resources closes the BufferedReader automatically even if an exception occurs, because the resource is closed before any catch block runs.

Test	Change / input	Expected behaviour	Actual result	Done
T1	marks file contains ICT001, 78	Valid mark accepted	The record was accepted and displayed. Import attempt finished printed.	☑
T2	ICT002, ABC	Invalid numeric value reported safely	NumberFormatException was caught, the line was skipped with a friendly message, and the program continued.	☑
T3	ICT001, 65 after ICT001, 78	Duplicate student ID identified	DuplicateStudentException was thrown and caught; the duplicate was skipped and reported.	☑
T4	ICT004, 120	Out-of-range mark rejected	IllegalArgumentException was thrown for 120 and caught; the record was rejected.	☑
T5	Filename changed to absent.txt	Friendly file-not-found message; finally runs	FileNotFoundException caught; friendly message shown; finally block printed "Import attempt finished".	☑

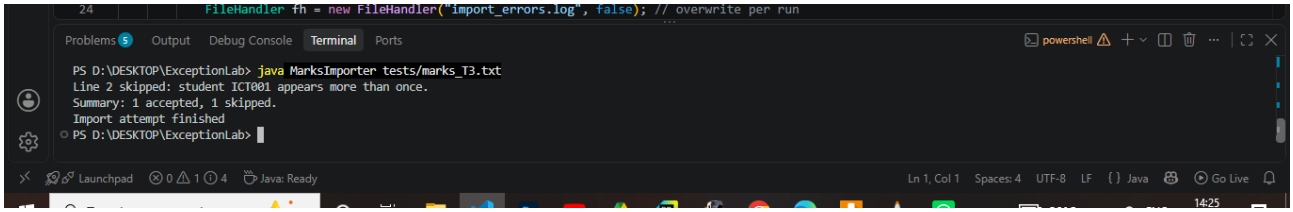
T1 console output



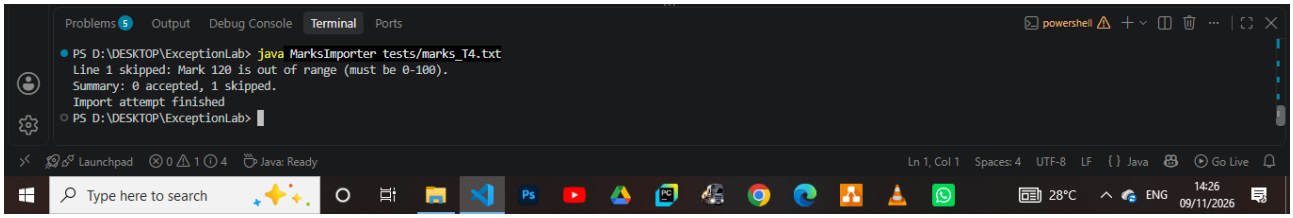
T2 console output



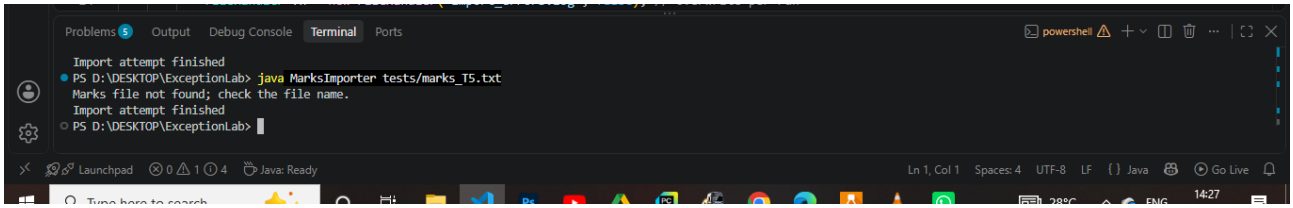
T3 console output



T4 console output



T5 console output



Full run on the supplied marks.txt

```
> java MarksImporter marks.txt
Accepted: ICT001 -> 78
Line 2 skipped: the mark is not a valid number.
Line 3 skipped: student ICT001 appears more than once.
Accepted: ICT003 -> 90
Summary: 2 accepted, 2 skipped.
Import attempt finished
```

Log file evidence (import_errors.log, T5 and T2 excerpts)

```
Nov 02, 2026 10:03:20 AM MarksImporter main
SEVERE: java.io.FileNotFoundException: absent.txt (No such file or directory)
Nov 02, 2026 10:03:16 AM MarksImporter importMarks
WARNING: Line 2 [ICT002, ABC] java.lang.NumberFormatException: For input string: "ABC"
```

Activity D: Engineer's review (5 marks)

1. Which of your exceptions is checked, and where does throws appear? Why?

FileNotFoundException/IOException is the checked exception. throws appears on the method signature: static void importMarks(String filename) throws IOException. The compiler forces this because file problems (missing file, read failure) are outside the program's control, so the caller must either handle

them or pass them on. My DuplicateStudentException, NumberFormatException and IllegalArgumentException are unchecked (RuntimeException subclasses), so they do not need throws.

2. Difference between throw and throws.

throw is a statement that actually raises an exception object at run time, e.g. throw new DuplicateStudentException(id); inside the loop. throws is part of a method declaration that warns callers that the method may pass on a checked exception, e.g. static void importMarks(String filename) throws IOException.

3. Why should the user receive a friendly message while the logger keeps technical details?

Ordinary users cannot act on stack traces and class names; a message like "Marks file not found; check the file name" tells them what to do. Technical details such as the exception type, message and stack trace are kept in the log for developers to diagnose the fault, and keeping them off the screen also avoids leaking internal information about the system.

4. What would go wrong with an empty catch block, or catching Exception before FileNotFoundException?

An empty catch block swallows the error: the program carries on as if nothing happened, the problem is never reported or logged, and bugs become very hard to find. Catching Exception before FileNotFoundException causes a compile error ("exception FileNotFoundException has already been caught") because the general catch makes the more specific one unreachable, and it would also hide the specific friendly handling. Specific exceptions must be caught first.